

Memoria técnica breve – Práctica 3

Aplicación de patrones de diseño en Django

David Manuel García Murillo

24 de abril de 2026

1. Resumen del proyecto

En esta práctica se amplió una aplicación web de gestión de eventos desarrollada con Django, incorporando nuevos patrones de diseño y manteniendo compatibilidad con los patrones ya implementados en prácticas anteriores.

La aplicación permite crear eventos de distintos tipos (conferencia, boda, concierto, cena), clonarlos, exportarlos y aplicar comportamiento dinámico en tiempo de ejecución.

2. Patrones aplicados

2.1. Decorator

Se implementó para añadir extras opcionales a un evento sin modificar su clase base. Ejemplos de extras: música en vivo, catering premium, seguridad VIP y streaming avanzado.

Beneficio: alta extensibilidad; se pueden añadir nuevos extras creando nuevos decoradores.

2.2. Observer

Se utilizó para notificar automáticamente a distintos módulos cuando cambia el estado o la fecha de un evento. Observadores implementados: notificación por email, proveedor y analytics.

Beneficio: desacopla la lógica de notificación del núcleo del evento.

2.3. Template Method

Se definió una plantilla común para el proceso de creación de eventos:

- 1) Validar datos
- 2) Configurar servicios

- 3) Calcular costes
- 4) Confirmar evento

Cada tipo de evento redefine pasos específicos.

Beneficio: uniformidad del flujo de negocio y reutilización de lógica común.

3. Integración con patrones previos

Además de los anteriores, el proyecto mantiene:

- **Builder:** construcción de eventos por tipo.
- **Prototype:** clonación de eventos.
- **Singleton:** configuración global (moneda/impuestos).
- **Chain of Responsibility:** validaciones previas.
- **Bridge:** salida HTML o JSON.
- **Adapter/Composite:** integración y composición de servicios.

4. Complicaciones encontradas

4.1. 1) Imports rotos por reorganización

Al mover archivos a subpaquetes (creacionales, estructurales, comportamiento) aparecieron errores de importación como: `ModuleNotFoundError: No module named 'patrones'`.

Solución: se reemplazaron imports antiguos por rutas correctas, preferentemente relativas dentro del paquete.

4.2. 2) Import circular

Apareció un ciclo entre `entidades.py` y `builder.py` al usar imports globales en `patrones/__init__.py`.

Solución: se evitó importar todo con `*` en el `__init__` raíz y se retiró el import de `builder` desde `creacionales/__init__.py`, rompiendo el ciclo.

4.3. 3) Error de sesiones en Django

Se produjo `OperationalError: no such table: django_session` al usar `request.session`.

Causa: la base de datos estaba configurada en memoria (`:memory:`), por lo que las migraciones no persistían.

Solución: configurar SQLite a archivo:

```
DATABASES['default']['NAME'] = BASE_DIR / 'db.sqlite3'
```

y ejecutar `python manage.py migrate`.

5. Conclusión

La práctica queda correctamente integrada y funcional: el sistema es más mantenible, modular y extensible. El uso combinado de patrones permite separar responsabilidades, reducir acoplamiento y facilitar futuras ampliaciones del proyecto.